

Best Practices for Claude

<https://code.claude.com/docs/en/best-practices>

General Caveat: As context window fills (from the conversation), LLM performance degrades.

General principles

- Give Claude a way to verify its work
 - Provide verification criteria, like telling Claude to write a test suite. Make sure the test suite makes sense
- Explore first, then plan, then code
 - Ask Claude to produce a detailed implementation plan (steps, what changes)
- Small changes can be implemented directly, to avoid plan mode overhead
- Provide specific context in your prompts
- Scope precisely, point to sources, reference existing patterns

Configure the environment

- Configure the environment
- Write an effective Claude.md file (short, human readable)
 - Global instructions go into ~/.claude/Claude.md, local ones in the project repository
 - Run /context to check the file is loaded
- Use the CLI
 - Install the GitHub (gh) CLI
- Connect MCP servers to use features from other tools
- Set up hooks to run scripts
 - Claude can write them for you
- Create skills
 - Claude invokes them automatically, they don't bloat the context
 - They can establish specific conventions, run workflows etc.
- Create custom subagents with permission to use specific tool sets
- Install plugins

Communicate effectively. Ask Claude the questions you would ask another engineer

- Ask codebase questions, use Claude for onboarding and learning
- Let Claude interview you, develop a spec interactively.

- Once the spec is done, start a new session for implementation

Manage your session effectively

- Sessions are persistent and reversible, use this to your advantage
- Course correct often and early. Too many failed attempts clutter the context, run /clear and start anew with a better prompt
- Manage context aggressively with /clear
- Instruct Claude to use subagents for research to avoid bloating the context
- Use checkpointing with /rewind
- Use —continue or —resume to resume conversations

Automate and scale once you are effective with one Claude session

- Run non-interactive session to integrate with CD/CI pipelines or other automated workflows
- Run multiple sessions, fan out across files, run autonomously
- Add an adversarial review step (in a subagent)
- Use another LLM for review

Avoid common mistakes

- Kitchen Sink Sessions: digressions. Use /clear between unrelated tasks
- Correcting over and over. Start fresh after /clear
- Overspecifying Claude.md - half the instructions get lost
- Trust-then-verify gap: Always include tests and provide verification
- Infinite exploration: Narrow the scope

Develop your own intuition!